

ColorPiano

ColorPiano is a Unity application that turns MIDI input into visual paint strokes on a curved canvas. Notes can come from a live MIDI device or from a MIDI file, and every note is converted into a brush stroke whose size, color, opacity, and brush shape are driven by MIDI data.

The rendering path is intentionally split into three parts:

1. A MIDI event source creates note-on and note-off events.
2. `CanvasVisualizer` receives those events and turns them into paint commands.
3. `CanvasPainter.compute` draws the paint commands into a `RenderTexture`.

The visible canvas itself is created by `SeamlessCanvas.cs`, which generates a wrapped mesh so the painting appears on a continuous surface instead of a flat quad.

Overview

What the Unity app does:

- listens to MIDI note input,
- maps each note to a visual stroke,
- chooses a brush based on the note height,
- calculates a color from pitch class, octave, and velocity,
- paints the result into a texture using a compute shader,
- displays that texture on the canvas mesh.

How it does it:

- `MidiBridge.cs` collects live note events from the MIDI input system.
- `MidiFilePlayer.cs` can load a MIDI file, schedule the events in time, and feed them into the same note pipeline.
- `CanvasPainter.compute` uses the note data to splat brush alpha into a render texture.
- `SeamlessCanvas.cs` builds the mesh that displays the texture in the scene.

1. MidiBridge.cs

`MidiBridge.cs` contains the main visualizer logic in `CanvasVisualizer`.

Canvas creation

The canvas is not a UI element. It is a 3D mesh surface created by `SeamlessCanvas.cs` and rendered with a material that reads from `canvasRT`.

Inside `CanvasVisualizer.Start()` the script prepares the drawing target:

- it finds the compute shader kernels,
- it references the canvas sphere component,
- it computes a radius scale factor,
- it clears the render texture to black.

The actual canvas texture is the `RenderTexture` assigned to `canvasRT`. That texture is the paint target used by the compute shader, and it is the image that gets shown on the canvas mesh.

How the texture for the canvas is created

The texture is not procedurally created in code. It is expected to be assigned in the Inspector as a `RenderTexture` asset. At runtime:

- `canvasRT` is cleared in `Start()` with `GL.Clear(...)`,
- each MIDI event writes directly into `canvasRT`,
- `ApplyCanvasFade()` gradually fades the existing content,
- `CanvasPainter.compute` updates the pixels every frame.

This means the canvas behaves like a paint buffer that keeps accumulating and fading strokes over time.

How brushes are selected and implemented

Brush selection is done in `GetBrushForNote(int noteNumber)`.

The mapping works like this:

- `lowestMappedNote` and `highestMappedNote` define the note-height range,
- the MIDI note number is clamped into that range,
- the clamped value is normalized to a 0 to 1 range,
- that normalized value is converted into an index into `heightBrushes`,
- the selected texture is returned,

- if the selected slot is empty, the script falls back to `brushMask`.

In practice, lower notes pick earlier brushes and higher notes pick later brushes. If `heightBrushes` is empty, the project behaves like the original single-brush version.

The brush texture is passed into `DrawOnCanvas(...)`, which forwards it to the compute shader as `brushTexture`.

How colors are calculated

Color generation is handled by `GetNoteColor(int noteNumber, float velocity)`.

The color model is:

- pitch class determines hue,
- octave influences brightness,
- velocity influences saturation and final intensity.

The script uses a circle-of-fifths style hue mapping:

- `noteNumber % 12` gives the pitch class,
- `((pitchClass * 7) % 12) / 12f` produces the hue,
- the octave increases perceived brightness,
- velocity pushes the color toward stronger, brighter output.

The final color is created with `Color.HSVToRGB(...)` and then multiplied to get a stronger HDR-looking result.

1.1 Brush and note flow in code

The important runtime path in `CanvasVisualizer` is:

1. a note event arrives,
2. a `BrushState` is created or updated,
3. a brush texture is selected from the note height,
4. a color is computed from the note value,
5. the compute shader writes the stroke into `canvasRT`.

That same path is used for both note-on and sustained note updates.

1.2 MidiFilePlayer.cs

`MidiFilePlayer.cs` is the MIDI file playback layer.

What it does

This script loads a `.mid` file from a `TextAsset`, converts it into scheduled note events, and sends them into `CanvasVisualizer` using `EnqueueMidiNote(...)`.

How files are loaded

The load path is:

1. `LoadFromMidiFile(...)` is called.
2. `TryBuildScheduleFromMidi(...)` creates a `MidiLoad` instance.
3. `MidiLoad.MPTK_Load(asset.bytes)` reads the MIDI bytes.
4. The loaded MIDI events are converted into `MidiScheduledNote` entries.
5. The events are sorted by time and played back in `Update()`.

Libraries used here

This script uses:

- `MidiPlayerTK` for `MidiLoad`, `MPTKEvent`, `MPTKCommand`, and `MidiStreamPlayer`,
- `UnityEngine` for timing and gameplay flow,
- standard C# collections for the schedule and active note tracking.

Event scheduling

Each MIDI event is converted into a lightweight struct:

- `timeSeconds` stores when the event should happen,
- `noteNumber` stores the MIDI pitch,
- `velocity` is normalized to 0 to 1,
- `isNoteOn` marks note-on versus note-off.

`Update()` advances playback by comparing the elapsed time against the next scheduled event. When an event is due, it is forwarded to `CanvasVisualizer` and optionally to the audio player.

Audio playback

If audio playback is enabled, the script also sends matching events to `MidiStreamPlayer`.

- note-on events are converted into `MPTKEvent` note-ons,
- note-off events stop the active note if it exists,
- if a matching active note is not found, a note-off event is sent directly.

1.3 CanvasPainter.compute

`CanvasPainter.compute` is the pixel-level drawing engine.

What the script does

The compute shader receives:

- a draw position,
- a color,
- a brush radius,
- an opacity value,
- a brush texture.

Then it writes into the `Result` render texture.

How brush painting works

`DrawSplat` does the following:

- calculates the pixel distance from the current fragment to the brush center,
- converts that distance into UV space,
- samples the brush alpha from `brushTexture`,
- uses the alpha and opacity to compute a paint weight,
- blends the new color with the existing pixel color using `lerp(...)`.

The brush texture is therefore not just a decorative image. Its alpha channel defines the stroke shape.

How fading works

`FadeCanvas` slowly reduces the brightness of the current canvas contents.

- `fadeAmount` controls how much the frame fades,
- `minCanvasOpacity` prevents the canvas from disappearing completely,
- the shader keeps some minimum visible color so the painting does not vanish instantly.

External libraries used

The project uses these external libraries and packages:

- [Unity Input System](#) used in `MidiBridge.cs` for device detection and MIDI device callbacks.
- [Minis by Keijiro](#) used in `MidiBridge.cs` for live MIDI input through `MidiDevice` and `MidiNoteControl`.
- [MidiPlayerTK](#) used in `MidiFilePlayer.cs` for MIDI file loading, scheduling, and optional audio playback. The package is included in the project under `Assets/MidiPlayer`.
- XR Toolkit for VR locomotion

Code references

- [Assets/Scripts/MidiBridge.cs](#)
- [Assets/Scripts/MidiFilePlayer.cs](#)
- [Assets/CanvasPainter.compute](#)

Practical setup summary

To make the full system work in the scene:

1. Create a canvas
2. Assign a `RenderTexture` to `canvasRT` in `CanvasVisualizer`.
3. Assign a compute shader asset to `painterShader`.
4. Add one fallback brush to `brushMask` or populate `heightBrushes` for height-based brush selection.
5. If you want MIDI file playback, assign `MidiFilePlayer.target` to the `CanvasVisualizer` instance and provide a MIDI file as a `TextAsset`.

Notes

- `heightBrushes` lets you use several brush textures and map them to note height.
- `brushMask` remains the fallback when no height-mapped brush is available.
- Velocity affects both stroke intensity and stroke radius.
- Color and brush selection are independent, so you can tune them separately.